

Final Project

PharmaKG Explorer: Knowledge Graph Embedding-Driven Conversational Querying and Real Time Link Prediction in Drug-Gene-Disease Networks

Course No: INFO703

Course Name: Biological Database Management

Instructor: Dr. Jake Chen

Submission Date: 05.02.2025

Submitted by

Shehan Irteza Pranto

PhD Student, Biomedical Engineering,

Graduate Research Assistant, Biomedical Informatics and Data Science

Research Fellow, Division of Nephrology, Department of Medicine

University of Alabama at Birmingham

Email: prantos@uab.edu | spranto@uabmc.edu



Title: PharmaKG Explorer: Knowledge Graph Embedding–Driven Conversational Querying and Real-Time Link Prediction in Drug-Gene-Disease Networks

Author: Shehan I. Pranto

Abstract: In this project, we develop an end-to-end platform for interactive knowledge graph embedding tailored to accelerate drug discovery. Starting from curated biomedical datasets-PharmAlchemy, we extract and filter 311496 triplets linking drugs, genes, diseases, and side effects, then deploy the complete graph including nodes, relations, and properties on Neo4j Aura. A natural-language interface built with Gradio and LangChain leverages LLM prompting (Gemma2-9b-It) to translate user inputs into Cypher queries, enabling seamless, conversational exploration of the graph. We also implemented and compared three state-of-the-art embedding methods - TransE, RotatE, and ComplEx- on link prediction benchmarks, evaluating performance via metrics such as Hits@k and mean reciprocal rank. To make link prediction findings actionable, we also provide an interactive GUI that allows researchers to visualize and test potential new relationships in real time. This integrated framework combines powerful embedding techniques with user-friendly interfaces to support rapid hypothesis generation and discovery in biomedical research.

1. Introduction:

The rapid expansion of biomedical data has transformed drug discovery into a data-intensive endeavor, yet the integration and interrogation of heterogeneous information sources remain significant challenges [1]. Traditional approaches to mining relationships among drugs, genes, diseases, and side effects often rely on manual curation or siloed analytical pipelines, which can impede hypothesis generation and slow the pace of translational research. Knowledge graphs structured representations of entities and their interrelations offer a powerful solution by unifying disparate datasets into a coherent, navigable network [2-3]. However, the sheer scale and complexity of biomedical knowledge graphs demand automated methods for discovering latent patterns and predicting novel links.

Knowledge graph embedding (KGE) techniques address this need by mapping entities and relations into a continuous vector space, where geometric operations capture semantic regularities. Models such as TransE, RotatE, and ComplEx have demonstrated strong performance on link prediction benchmarks, enabling researchers to infer missing edges that correspond to plausible drug–gene, gene–disease, or drug–disease associations [4]. Various knowledge graph embedding (KGE) techniques, including translation-based and bilinear models, have been developed to map entities and relations into continuous vector spaces, facilitating the prediction of missing links in drug-gene-disease networks [5]. Recent models incorporate attention mechanisms to enhance feature extraction and improve screening accuracy, as demonstrated in drug repositioning applications for COVID-19 [6]. Despite these advances, there remains a gap between embedding algorithms and end users: biologists and clinicians often lack the computational expertise to formulate complex queries or interpret high-dimensional embeddings [7]. Bridging this gap

requires an integrated platform that not only computes embeddings at scale but also presents results through intuitive, interactive interfaces.

In this project, we present **PharmaKG Explorer**, an end-to-end system that combines state-of-the-art KGE methods with conversational querying to facilitate drug discovery workflows. We begin by ingesting curated biomedical triplets drawn from databases such as PharmAlchemy into a Neo4j Aura graph database, enriching nodes with domain-specific properties. A Gradio-based front end, augmented by LangChain and a fine-tuned **Gemma2-9b-It** model, translates user questions into optimized Cypher queries, allowing even non-expert users to navigate the graph via natural-language prompts. Under the hood, embeddings generated by TransE, RotatE, and ComplEx support link prediction tasks; predictions are immediately rendered in the UI, enabling researchers to test hypotheses and iteratively refine their queries. Among the evaluated models, **ComplEx** worked best achieving the highest mean reciprocal rank and all the Hits@k (e.g., k=1,10,50,100) scores for tail prediction.

System Architecture & Key Components

This work encompasses six key components:

1. **Triplet Construction:** assembling high-quality drug–gene–disease triplets for graph building
2. **Graph Deployment:** loading nodes, relations, and properties into Neo4j Aura
3. **Conversational GUI:** interacting with Neo4j via a Gradio interface that converts natural-language input into Cypher via LLM prompting and displays results
4. **Embedding Engine:** training TransE, RotatE, and ComplEx models on the graph
5. **Evaluation Framework:** comparing embeddings on held-out triplets using link prediction metrics
6. **GUI for Link Prediction:** enabling real-time hypothesis testing through the GUI

Through this integrated framework, PharmaKG Explorer democratizes access to advanced graph analytics and accelerates data-driven hypothesis generation in biomedical research.

2. Methodology:

2.1 Data Processing and Triplets Construction:

To ensure high-quality input for knowledge graph embedding (KGE) and downstream link prediction, we designed a modular and scalable data preprocessing pipeline that transforms raw PharmAlchemy datasets into structured RDF-style triplets. This section outlines the filtering of raw data, construction of relation-specific triplets, and preparation of a unified triplet dataset for embedding training.

2.1.1 Dataset Filtering ([data filtering.py](#)):

We began with four core datasets containing biomedical associations:

- R2G: drug–gene interactions,

- R2D: drug–disease associations,
- D2G: disease–gene associations, and
- G2G: gene–gene relationships.

We applied a systematic approach to retain only biologically relevant and validated entries. This process involved:

a. Identifier Normalization

All CIDs (PubChem compound IDs), UMLS CUIs (Concept Unique Identifiers), and gene names were cast to string format and normalized (uppercase, whitespace-trimmed) to ensure consistency across datasets.

b. Drug Filtering

Only drugs present in `R2G.csv` were retained in the R2D dataset, reducing noise and ensuring relevance.

c. Disease and Gene Filtering

For the D2G dataset, only diseases with validated drugs (from R2D) were considered. Similarly, the G2G dataset was filtered to include only gene pairs where at least one gene appeared in either the validated drug or disease sets.

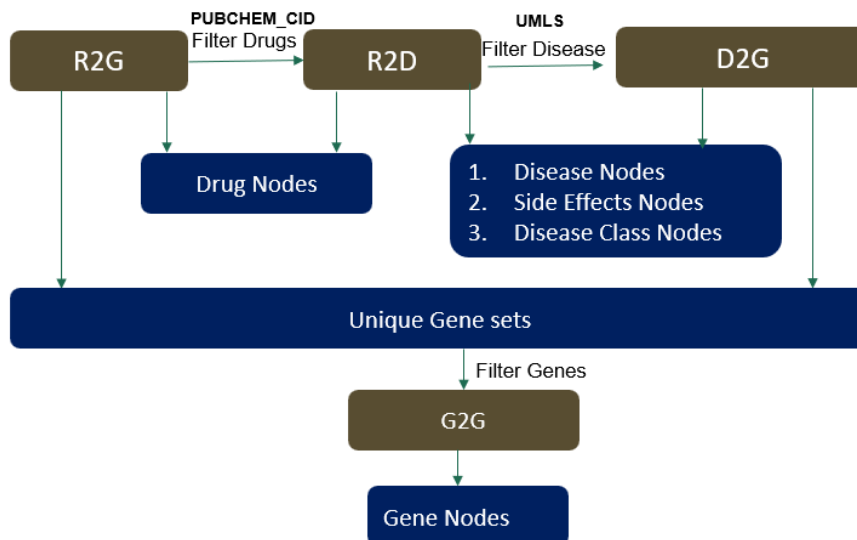


Figure 1: Flow Diagram of filtering raw data and extracting entities/notes for KG

2.1.2 Triplets Construction ([triplets_builder.py](#)):

The pipeline transforms filtered data into RDF-like triples in the form ⟨subject, predicate, object⟩. Each dataset contributed triplets with distinct semantic structures:

Table 1. Triplet Templates Constructed from Curated Data Sources

Dataset	Triplet Types
R2G_filtered	Drug → interacts_with → Gene
R2D_filtered	Drug → treats → Disease Drug → has_side_effect → Side Effect
D2G_filtered	Gene → associated_with/<semantic type> → Disease Gene → associated_with → Disease Disease → has_class → Disease_class
G2G_filtered	Gene → associated_with → Gene

Table 2. Quantitative Overview of Knowledge Graph Construction Input

Source Dataset	Triplets Generated	Unique Subjects	Unique Objects	Unique Predicates
R2G	32162	7968	2509	26
R2D	7208	564	1772	2
D2G	24660	7609	814	13
G2G	247466	1370	15643	1
Total	311496	17511	20738	42

2.2 Graph Database(Neo4j Aura) Deployment ([neo4j_upload.py](#)):

To enable efficient graph exploration, querying, and downstream embedding, we deployed the structured biomedical triplets into a cloud-hosted graph database using Neo4j Aura. The database stores entities such as drugs, genes, and diseases as labeled nodes, and their relationships as typed edges. This structure facilitates semantic querying, visualization, and integration with natural language interfaces.

2.2.1. Upload Pipeline Overview:

We prepared a Python module ([neo4j_upload.py](#)) to automate the ingestion of triplets into Neo4j. The function `upload_to_neo4j()` accepts a Pandas DataFrame containing columns ['subject', 'predicate', 'object'], and uploads each triple as two nodes connected by a relationship. The procedure consists of:

a. Session Initialization:

A Neo4j driver is instantiated using secure credentials provided by the user.

b. Graph Cleanup:

All existing nodes and relationships are deleted via the Cypher command:

```
MATCH (n) DETACH DELETE n
```

c. Triplet Parsing and Insertion:

Each predicate is parsed using / as the delimiter. The segments define:

- Node labels (e.g., Drug, Gene, Disease)
- Relationship type (e.g., INTERACTS_WITH, TREATS)
- Optional edge property (e.g., inhibits, activates)

```
MERGE (a:Drug {name: "Ibuprofen"})  
MERGE (b:Disease {name: "Inflammation"})  
MERGE (a)-[r:TREATS]->(b)  
SET r.property = "anti-inflammatory"
```

d. Execution:

For each row in the DataFrame, the above query is dynamically constructed and executed. Upon completion, a message confirms successful graph upload.

2.2.2. Nodes and Relationship Schema:

The database schema is flexibly inferred from the predicate structure. For example:

Table 3. Mapping of Predicate Structure to Node Labels, Relationship Types, and Properties

Predicate Example	Node A Label	Relation Type	Edge Property	Node B Label
drug/interacts_with/ inhibits/gene	Drug	INTERACTS_WITH	inhibits	Gene
gene/ associated_with/ DISEASE_OR_SYNDROME/ disease	Gene	ASSOCIATED_WITH	DISEASE_OR_SYNDROME	Disease
disease/has_class/disease_class	Disease	HAS_CLASS	–	Disease_class

2.3. Knowledge Graph Embedding Models ([train_models.py](#)):

To model the semantic structure of the biomedical knowledge graph and uncover latent relationships between drugs, genes, and diseases, we trained and evaluated three prominent knowledge graph embedding (KGE) models: **TransE**, **ComplEx**, and **RotatE**. These models were implemented using the AmpliGraph library, which supports a wide range of multi-relational learning techniques tailored for link prediction. The key idea behind KGE is to embed both entities and relations into a continuous vector space such that valid triplets ((subject, predicate, object)) are assigned higher plausibility scores than invalid ones. These embeddings provide the foundation for downstream tasks such as discovering novel drug targets, suggesting repurposing candidates, or expanding gene–disease associations.

Embedding Workflow Overview:

Figure 2. shows the overall process of training knowledge graph embeddings. It starts with a knowledge graph, where each entity and relation is converted into a vector using a lookup layer. These vectors are passed to a scoring function $f(s, p, o)$ that measures how likely a triplet is to be true. During training, negative samples are created by corrupting the original triplets, and a loss function compares the scores of true and false triplets. The model updates embeddings to reduce this loss. After training, the model can be used for link prediction, helping discover missing or new relationships in the graph.

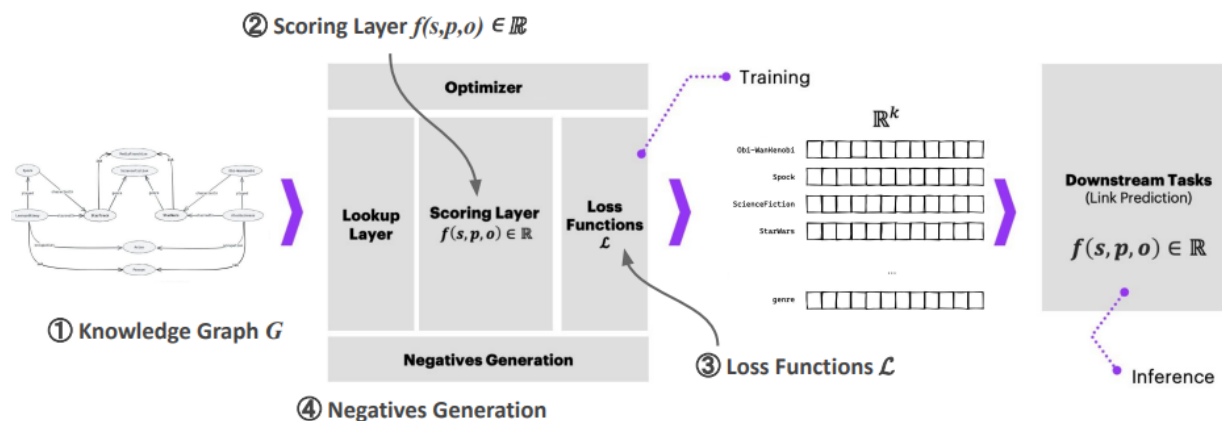


Figure 2: Embedding workflow Overview

2.3.1. Data Preparation and Training Setup:

We used the full triplet dataset (**all_triplets.csv**) and applied a 3-way split using AmpliGraph's `train_test_split_no_unseen()` function. This ensured that all entities appearing in the validation and test sets were also seen during training, thereby avoiding unseen-entity bias. Specifically, 500 triplets were reserved for validation and 1,000 for testing. The remainder was used for

training. A filter_positives dictionary containing all true triplets was used to eliminate known positives from negative sampling during evaluation.

All models were trained using a unified configuration. The table below outlines the shared training hyperparameters across all three KGE models:

Table 4. Common Training Parameters for KGE Models		
Parameter	Value	Explanation
k	250	Embedding dimensionality; each entity/relation is a 250-dimensional vector.
Optimizer	Adam	Adaptive-momentum optimizer for efficient gradient updates.
— learning_rate	1×10^{-3}	Step size for each update.
Loss	Pairwise margin	Margin-ranking loss: pushes positive triples closer than negatives by margin γ .
— margin	0.5	It demands that the model assigns the positive triple a score at least γ higher than the negative triple.
Regularizer	LP	Adds an L_p penalty on embeddings to prevent overfitting.
— p	2	Uses L_2 (Euclidean) norm.
— λ	1×10^{-5}	Regularization strength.
Training epochs	100	Maximum epochs; early stopping will usually end training sooner.
Batch size	len(train)/10	Each gradient step uses one-tenth of the training set.
Validation data	500	Held-out triples monitored during training.
EarlyStopping callback		
— monitor	val_hits10	Stops when validation Hits@10 stops improving.
— mode	max	Seeks to maximize Hits@10.
— patience	5	Allows up to 5 validations without improvement before halting.
restore_best_weights	True	Rolls back to the weights that achieved the best monitored metric.

2.3.2. Scoring Functions:

a. TransE: Translation-Based Embeddings:

TransE is a foundational embedding model that treats relationships as translation operations in Euclidean space. The core idea is to learn embeddings such that for a valid triplet $\langle s, r, o \rangle$:

$$\mathbf{e}_s + \mathbf{e}_r \approx \mathbf{e}_o$$

The scoring function penalizes deviations from this equality:

$$f(s, r, o) = \|\mathbf{e}_s + \mathbf{e}_r - \mathbf{e}_o\|_2$$

This model performs well on one-to-one relations but struggles with symmetric, one-to-many, or hierarchical relationships that frequently occur in biomedical graphs.

b. RotatE: Modeling Relations as Rotations:

RotatE introduces a geometric interpretation of relations as rotations in the complex vector space. It enforces the constraint that each relation $\mathbf{e}_r$ has unit modulus, acting as a rotation:

$$\mathbf{e}_o = \mathbf{e}_s \circ \mathbf{e}_r$$

Its scoring function is:

$$f(s, r, o) = -\|\mathbf{e}_s \circ \mathbf{e}_r - \mathbf{e}_o\|_2$$

where $\circ$ denotes element-wise complex multiplication. This allows the model to capture not just symmetry and asymmetry but also inverse and compositional relations, which are useful for modeling complex biological interaction chains.

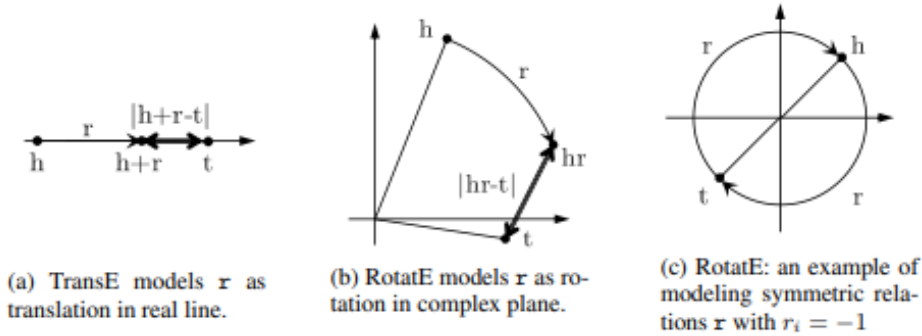


Figure 3 : Illustration of TransE and RotatE with only 1 dimension of embedding [8]

c. ComplEx: Complex-Valued Embedding for Asymmetry:

ComplEx extends embeddings into the complex number space, allowing it to model both symmetric and asymmetric relations. For a triplet $\langle s, r, o \rangle$, the scoring function is defined as:

$$f(s, r, o) = \text{Re}(\langle \mathbf{e}_s, \mathbf{e}_r, \bar{\mathbf{e}}_o \rangle)$$

Here, $\sim e_o$ is the complex conjugate of the object embedding, and $\text{Re}(\cdot)$ denotes the real part of the inner product. This model is particularly powerful for biomedical graphs where directionality (e.g., drug *treats* disease) is essential. In our experiments, ComplEx achieved the best performance across all ranking metrics.

2.4. Evaluation Metrics and Performance Comparison:

We evaluated the performance of all embedding models using standard **link prediction metrics**, namely **Mean Reciprocal Rank (MRR)** and **Hits@k**. **Mean Reciprocal Rank (MRR)**:

- **Mean Reciprocal Rank (MRR):**

The **MRR** measures the average inverse rank of the correct entity in a list of candidates generated by the model. For each test triplet, the model ranks all possible corrupted triplets (by replacing either the subject or object), and MRR is calculated as:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

where Q is the set of all test queries and rank_i is the position of the correct answer in the ranked list. A higher MRR indicates that the model tends to assign higher scores to correct predictions.

The **Hits@k** metric evaluates the proportion of times the correct entity appears in the top-k ranked predictions. For instance, **Hits@10** shows how often the true entity is among the top 10 predictions. We report Hits@1, Hits@10, Hits@50, and Hits@100 to capture both strict and lenient retrieval performance.

2.4.1. Test 1: Performance comparisons of the scoring function:

	Model	MRR	Hits@1	Hits@10	Hits@50	Hits@100
0	ComplEx	0.139760	0.0820	0.2480	0.4565	0.5500
1	RotatE	0.090654	0.0475	0.1645	0.3160	0.4025
2	TransE	0.048397	0.0030	0.1340	0.3035	0.4085

Table 5. Model Performance on link (tail) prediction

These results demonstrate that **ComplEx consistently outperforms** TransE and RotatE across all metrics. Its superior performance is likely due to its capability to handle non-commutative, asymmetric relationships which are prevalent in biomedical datasets.

2.4.2. Test 2: Tail Prediction Case Study: Model Ranking Comparison

We evaluated model performance on 10 randomly selected test triplets by fixing the head and relation and ranking all possible tail candidates. Each model -ComplEx, RotatE, and TransE scored these candidates, and the true tail's rank was recorded. Known true triplets were filtered out during scoring to ensure fairness.

	Head	Relation	True Tail	Rank (ComplEx)	Rank (RotatE)	Rank (TransE)
0	CARPHENAZINE	drug/interacts_with/antagonist/gene	DRD1	1	1	8
1	KDM6A	gene/associated_with/NEOPLASTIC_PROCESS/disease	NEOPLASMS	6	12	61
2	EEF1A2 EEF1AL STN	gene/associated_with/gene	TUBB1	50	64	736
3	CHLORDIAZEPOXIDE	drug/interacts_with/gene	AR	14	4392	137
4	OLFM4 GW112 UNQ362/PRO698	gene/associated_with/gene	TFAP2B	89	219	415
5	CCND1 BCL1 PRAD1	gene/associated_with/gene	RFC4	237	31	19
6	RIN3	gene/associated_with/gene	GOLGA7 GCP16 HDCKB03P HSPC041	7438	217	5838
7	COL1A1	gene/associated_with/gene	TGFB1 TGFB	5	10	9
8	VCL	gene/associated_with/gene	MSN	3	524	4
9	TRICHOSTATIN	drug/interacts_with/gene	SIRT2	522	6879	1041

Table 6. Model Comparison on Tail Prediction for Selected Triplets

Findings:

- ComplEx consistently ranked the true tail entity higher than RotatE and TransE in most cases.
- In the CARPHENAZINE–antagonist–DRD1 example, ComplEx and RotatE both ranked the correct tail at position 1, while TransE ranked it 8th.
- RotatE's performance was inconsistent across cases, showing large variations in ranks (e.g., ranks 12, 31, and 524).
- TransE underperformed overall, especially in complex or multi-relational examples like KDM6A–NEOPLASMS (ranked 61 vs. 6 by ComplEx).
- Challenging cases like OLFM4–TFAP2B showed that even ComplEx struggled (rank 89), suggesting that sparse relations remain difficult to model.
- Despite some exceptions, ComplEx demonstrated more stable and accurate rankings overall.
- These results support the use of ComplEx for biomedical knowledge graphs with asymmetric and context-specific relationships.

3. User Interfaces for Knowledge Graph Interaction:

This section describes the two interactive components developed to facilitate seamless exploration and hypothesis testing on the knowledge graph: a Conversational Interface for natural language querying to NEO4J, and a Real-Time Link Prediction UI for visualizing link prediction results powered by embedding models.

3.1. Conversational Interface ([GUI neo4j_graph_explorer.py](#)):

The Conversational Interface is a web-based Graph Explorer built using Gradio for the front end and LangChain for natural language processing. It enables users especially those without knowledge of Cypher query language to explore the knowledge graph by simply asking questions in plain English. The interface connects to both a Neo4j Aura graph database and a Groq-hosted LLM (e.g., Gemma2-9b-It) to support natural language query translation and real-time data retrieval.

Functionalities:

- 1. Natural Language Querying:** Users can ask biomedical questions in plain English (e.g., “What drugs treat epilepsy?”), without needing Cypher knowledge.
- 2. Automatic Cypher Generation:** Queries are translated into valid Cypher statements using a Groq-hosted LLM (e.g., Gemma2-9b-It) via LangChain's GraphCypherQChain.
- 3. Real-Time Graph Interaction:** Generated Cypher queries are executed live on a Neo4j Aura instance, and the results are returned instantly to the user.
- 4. LLM Prompting with Schema Awareness:** The LLM is guided by a custom system prompt that defines the Neo4j graph schema specifying node types (e.g., **Drug**, **Gene**, **Disease**) and relationships (e.g., **TREATS**, **ASSOCIATED_WITH**). This ensures that generated Cypher queries are structurally correct and contextually relevant to the biomedical domain.
- 5. Execution Transparency:** The full Cypher query and query logs are displayed alongside the result, helping users understand how their question was interpreted and processed.
- 6. Entity Filtering via Dropdown Menus:** Users can browse and select from categorized dropdown lists for: Genes, Drugs, Diseases, Side Effects, Disease Semantic Types
- 7. Disease Categorization Support:** Diseases can be filtered by their semantic type (e.g., "NEOPLASM", "CARDIOVASCULAR DISEASE"), making it easier to ask more specific and guided questions.
- 8. Interactive Result Display:** Query results are rendered in a clean format, with both technical logs and answers shown side-by-side.

Case Study:

To demonstrate the practical capabilities of the Conversational Interface, we present a case study using three diverse natural language queries. These queries showcase how biomedical users can interact with the graph to extract insights without needing to know Cypher syntax.

Query 1: Find the drugs with the highest number of side effects in the database.

Response from UI :

The screenshot displays the user interface of the Conversational Interface. It is divided into three main sections: 'Natural-Language Query', 'Generated Cypher', and 'Result Given by Neo4j'. The 'Natural-Language Query' section contains the text 'Find the drugs with the highest number of side effects in the database.' The 'Generated Cypher' section shows the corresponding Cypher query:

```
MATCH (d:Drug)-[:HAS_SIDE_EFFECT]->(se:Side_effect)
WITH d, count(se) AS num_side_effects
ORDER BY num_side_effects DESC
RETURN d.name, num_side_effects
LIMIT 10
```

 The 'Result Given by Neo4j' section displays the output: 'ALPRAZOLAM and (3R)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE and (3S)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE and FLUOXETINE have the highest number of side effects.'

Natural-Language Query

Query

Find the drugs with the highest number of side effects in the database.

Logs

> Entering new GraphCypherQChain chain...

Generated Cypher

Generated Cypher:

```
MATCH (d:Drug)-[:HAS_SIDE_EFFECT]->(se:Side_effect)
WITH d, count(se) AS num_side_effects
ORDER BY num_side_effects DESC
RETURN d.name, num_side_effects
LIMIT 10
```

Full Context:

[[{'d.name': 'ALPRAZOLAM', 'num_side_effects': 43}, {'d.name': '(3R)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE', 'num_side_effects': 42}, {'d.name': '(3S)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE', 'num_side_effects': 42}, {'d.name': 'FLUOXETINE', 'num_side_effects': 42}]]

Result

Result Given by Neo4j

ALPRAZOLAM and (3R)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE and (3S)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE and FLUOXETINE have the highest number of side effects.

Cross check the result in Neo4j by the generated cypher:

The screenshot shows the Neo4j GUI interface. At the top, the Cypher query is entered:

```
neo4j$ MATCH (d:Drug)-[:HAS_SIDE_EFFECT]->(se:Side_effect) WITH d, count(se) AS sideEffectCount ORDER BY sideEffectCount DESC RETURN d.name, sideEffectCount
```

 Below the query, there are two tabs: 'Table' and 'RAW'. The 'Table' tab is selected, and it displays a table with two columns: 'd.name' and 'sideEffectCount'. The table contains three rows of data, which are highlighted with a red box.

d.name	sideEffectCount
"ALPRAZOLAM"	43
"(3R)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE"	42
"(3S)-N-METHYL-3-PHENYL-3-[4-(TRIFLUOROMETHYL)PHENOXY]PROPAN-1-AMINE"	42

The generated Cypher was correct and worked in Neo4j aura, The given drug names with highest side effects were also matched with the results of the GUI

Query 2: What diseases are treated by ALPRAZOLAM?

Logs

> Entering new GraphCypherQChain chain...
Generated Cypher:
MATCH (d:Drug {name: "ALPRAZOLAM"})-[:TREATS]->(disease:Disease) RETURN disease.name

Result

ANXIETY DISORDERS, PANIC DISORDER, PANIC ATTACKS, ACHE, SUBSTANCE ABUSE PROBLEM, DEPRESSION, NEUROTIC, SLEEPLESSNESS, DEPRESSIVE SYMPTOMS, TACHYCARDIA, FLUSHING

Query 3: What genes are associated with ALPRAZOLAM?

Logs

> Entering new GraphCypherQChain chain...
Generated Cypher:
MATCH (d:Drug {name: "ALPRAZOLAM"})-[:INTERACTS_WITH]->(g:Gene) RETURN g.name

Full Context:
[{'g.name': 'TSPO'}, {'g.name': 'CREBBP'}, {'g.name': 'GABRA5'}, {'g.name': 'GABRA6'}, {'g.name': 'GABRB2'}, {'g.name': 'GABRB3'}, {'g.name': 'GABRD'}, {'g.name': 'GABRG1'}, {'g.name': 'GABRG2'}, {'g.name': 'GABRP'}]

> Finished chain.

Result

TSPO, CREBBP, GABRA5, GABRA6, GABRB2, GABRB3, GABRD, GABRG1, GABRG2, GABRP are associated with ALPRAZOLAM.

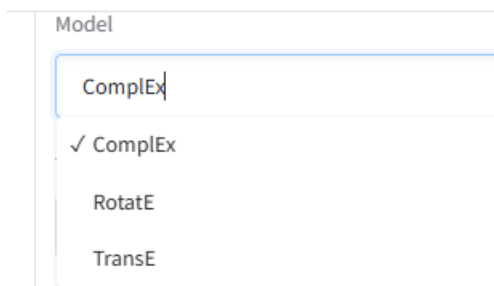
3.2. UI for Real-time link Prediction ([GUI KG link prediction.py](#)) :

The KG Link-Prediction Explorer is a Gradio-based interactive interface designed to let users perform real-time tail prediction using trained knowledge graph embedding (KGE) models: ComplEx, RotatE, and TransE. Upon launching, the interface loads a curated triplet dataset and restores pretrained models from .pkl files. Users can then select test triplets and observe how each model ranks the correct tail entity among all possible candidates. The UI supports both Top-K prediction and true-tail rank inspection, providing insights into each model's predictive capability on biomedical relations.

Functionalities:

- **Model Selection**

Users can select from trained KGE models (ComplEx, RotatE, TransE) for running predictions on the same test queries, enabling comparative evaluation.



- **Top-K Tail Prediction**
Given a head entity and a relation, the interface returns the top-K predicted tail entities ranked by model confidence scores.
- **True Tail Rank Reporting**
For each selected triplet, the UI computes and displays the rank of the true tail entity within the model's predicted candidate list.
- **Support for Multiple Triplet Selection**
Users can choose multiple test triplets at once to compute and compare tail ranks across all models.
- **Custom Data Loading**
At startup, users provide paths to the triplet CSV (all_triplets.csv) and pretrained model files (.pkl), ensuring flexibility across datasets and models.
- **Score Transparency**
For each Top-K prediction, the tail entity's confidence score is shown alongside its rank, helping users interpret model certainty.
- **Filtered Ranking**
The prediction function excludes known training and validation triplets from scoring to ensure fair and meaningful test evaluation.
- **Designed for Biomedical Tail Prediction**
The UI is tailored for biomedical KGs, allowing intuitive inspection of drug–gene–disease–side effect relationships and their ranked predictions.
- **Interactive Two-Tab Layout**
 - **Tab 1: Predict Tail:** Choose a single triplet and model to view Top-K predictions.
 - **Tab 2: True-Tail Ranks:** Select multiple triplets to compare true tail ranks across models.

Case Study:

Tab 1: Predict tail:

We want to see the top 10 ranks for predicted tails by ComplEx scoring function for a given test triplets. The model grabs the Head and relations and ranks the all available tails and also show the scores maintaining the sequence. Besides, the GUI shows the ranking of the True tail among the predicted tails.

Model

ComplEx

Test Triplet

CHEMBL599924 | drug/interacts_with/gene | POLH

Top K

10

Submit

Top-K Predictions

Rank	Tail	Score
1	KDM4A	1.7849688529968262
2	IDH1	1.7672978639602661
3	POLB	1.7006653547286987
4	POLI	1.6262110471725464
5	FEN1	1.5841667652130127
6	MAPT	1.5187780857086182
7	BAZ2B	1.5095160007476807
8	EHMT2	1.5042740106582642
9	CBX1	1.4941340684890747
10	POLH	1.4931546449661255

True-Tail Rank

True tail 'POLH' is at position 10 of 31379

Tab 2: True Tails Ranks:

We randomly selected a subset of test triplets to evaluate tail prediction performance across all trained models. For each triplet, the models were tasked with predicting the tail entity, and the rank of the true tail among all candidate entities was computed. The developed GUI complements this process by displaying the ranks of the true tails across all models in an interactive format. This enables users to visually compare model performance in terms of ranking accuracy, thereby supporting detailed evaluation and analysis of each model’s predictive

capabilities.

Head	Relation	True Tail	Rank (Complex)	Rank (RotatE)	Rank (TransE)
CARPHENAZINE	drug/interacts_with/antagonist/gene	DRD1	1	1	8
KDM6A	gene/associated_with/NEOPLASTIC_PROCESS/disease	NEOPLASMS	6	12	61
EEF1A2 EEF1A1 STN	gene/associated_with/gene	TUBB1	50	64	736
CHLORDIAZEPOXIDE	drug/interacts_with/gene	AR	14	4392	137
OLFM4 GW112 UNQ362/PRO698	gene/associated_with/gene	TFAP2B	89	219	415
CCND1 BCL1 PRAD1	gene/associated_with/gene	RFC4	237	31	19
RIN3	gene/associated_with/gene	GOLGA7 GCP16 HDCKB03P HSPC041	7438	217	5838

4. Limitations

While the developed platform demonstrates strong capabilities in biomedical knowledge graph querying and link prediction, several limitations remain:

- **Schema Dependency for LLMs:** The natural language interface relies heavily on predefined prompts and a static graph schema. Any structural changes in the Neo4j database require prompt updates and reinitialization.
- **Limited Generalization to Unseen Entities:** The current KGE models (Complex, TransE, RotatE) are trained on fixed entities. They cannot predict links involving entirely new nodes not present in training.
- **LLM Interpretation Challenges:** While LangChain + Groq-hosted LLMs generally generate accurate Cypher, they occasionally misinterpret ambiguous or compound user questions, leading to invalid or incomplete queries.
- **Sparse Relationship Modeling:** Even Complex, the best-performing model, struggles with sparse or weakly connected entities-common in biomedical datasets.

5. Future Directions:

To enhance the platform's robustness, scalability, and impact, we propose the following future enhancements:

- **Support for Zero-Shot and Inductive Learning:** Incorporate embedding methods (e.g., GNN-based or path-based) that can handle unseen entities during inference.
- **Prompt Optimization and Query Chaining:** Improve LLM prompt engineering to support multi-hop reasoning and compound biomedical queries.
- **Explainability Features:** Integrate modules to explain why a certain link was predicted e.g., through attention scores, related paths, or biomedical literature evidence.
- **Dynamic Schema Updates:** Automate schema ingestion for Neo4j so that the LLM can adapt to database structure changes without manual prompting.

- **Integration with External Knowledge Sources:** Extend the graph with data from DrugBank, PubChem, and clinical trials to broaden biological context.

6. Conclusion:

This project introduces PharmaKG Explorer, an integrated, interactive platform for biomedical knowledge graph exploration and link prediction. Using curated drug–gene–disease–side effect triplets, we trained and evaluated three embedding models TransE, RotatE, and ComplEx and deployed them in a real-time Gradio interface. The system also includes a conversational querying interface powered by a Groq-hosted LLM, enabling non-technical users to ask natural language questions and retrieve graph-based answers. Quantitative evaluation confirmed that ComplEx outperformed other models across all metrics (MRR, Hits@K), and qualitative tests showed its consistency in ranking true tail entities.

By combining natural language understanding, graph embeddings, and an accessible user interface, PharmaKG Explorer bridges the gap between biomedical researchers and complex graph analytics. It simplifies data access while enabling discovery of potential relationships useful in drug repurposing, gene prioritization, and disease association studies.

References:

- [1] Bang, Dongmin, et al. "Biomedical knowledge graph learning for drug repurposing by extending guilt-by-association to multiple layers." *Nature Communications* 14.1 (2023): 3570.
- [2] MacLean, Finlay. "Knowledge graphs and their applications in drug discovery." *Expert opinion on drug discovery* 16.9 (2021): 1057-1069.
- [3] Dive, Deep. "Using knowledge graphs to drive drug discovery Break knowledge silos to power R&D with data science."
- [4] Bonner, Stephen, et al. "Understanding the performance of knowledge graph embeddings in drug discovery." *Artificial Intelligence in the Life Sciences* 2 (2022): 100036.
- [5] Vora, Lalitkumar K., et al. "Artificial intelligence in pharmaceutical technology and drug delivery design." *Pharmaceutics* 15.7 (2023): 1916.
- [6] Albahli, Saleh, and Momina Masood. "Efficient attention-based CNN network (EANet) for multi-class maize crop disease classification." *Frontiers in Plant Science* 13 (2022): 1003152.
- [7] Vora, Lalitkumar K., et al. "Artificial intelligence in pharmaceutical technology and drug delivery design." *Pharmaceutics* 15.7 (2023): 1916.
- [8]. Sun, Zhiqing, et al. "Rotate: Knowledge graph embedding by relational rotation in complex space." *arXiv preprint arXiv:1902.10197* (2019).

Appendices:

Appendix A: Conversational Interface

Conversational Interface

Gene

Disease Semantic Type

Disease Name

Drug

Side Effect

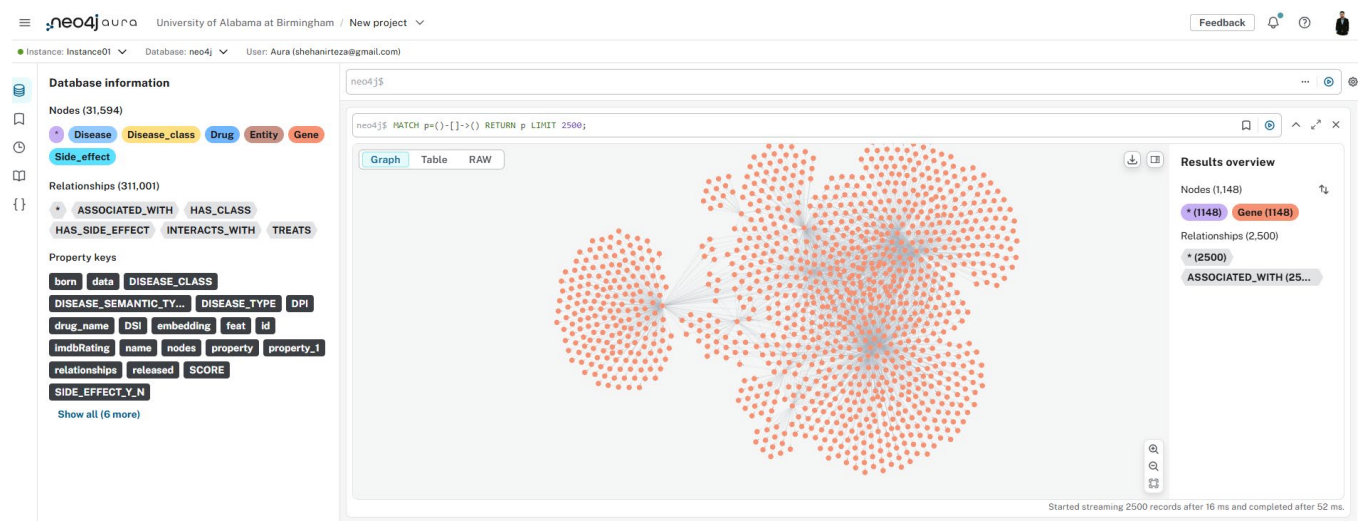
Natural-Language Query

Logs

Result

Ask Graph

Appendix B: Graph on Neo4j



Appendix C: KG link Prediction GUI

KG Link-Prediction Explorer

1. Predict Tail 2. True-Tail Ranks

Select triplet, enter K, click Submit

Model
ComplEx
Test Triplet
MT3 gene/associated_with/gene FAM162A C3ORF28 E2IG5 DC16 FWP001
Top K
10

Submit

Top-K Predictions

1	2	3
---	---	---

True-Tail Rank

KG Link-Prediction Explorer

1. Predict Tail 2. True-Tail Ranks

Multi-select and click Compute

Test Triplets

Compute Ranks

True-Tail Ranks

1	2	3